261026 pwm_motorcontrol_박수민

[pwm_duty_control.v]

```
`timescale 1ns / 1ps

// (1) 100MHz ---> 10MHz 의 주파수를 생성
// (2) 10MHz로 duty 조절 (0~9까지 총 10단계)
module pwm_duty_control(
    input clk,
    input reset,
    input [2:0] clean_btn,        // btn[0]=U btn[1]=
    output [3:0] duty_cycle,     // FND 출력
    output pwm_out,
    output pwm_out_led
    );

    reg [3:0] r_duty_cycle = 4'd5;  //default 50%
    reg [3:0] r_pwm_counter = 4'd0;
    reg r_prev_duty_inc = 1'b0;
    reg r_prev_duty_dec = 1'b0;

    // 0. btnU btnD rising edge 검출
    wire w_duty_inc = (clean_btn[0] && !r_prev_duty_inc);
    wire w_duty_dec = (clean_btn[1] && !r_prev_duty_dec);

    // 1. duty cycle 제어 (btnU btnD)
    always @(posedge clk, posedge reset) begin
        if (reset) begin
            r_duty_cycle <= 4'd5;
            r_prev_duty_inc <= 0;
            r_prev_duty_dec <= 0;
        end else begin
            r_prev_duty_inc <= clean_btn[0];
            r_prev_duty_dec <= clean_btn[1];
            if (w_duty_inc && r_duty_cycle < 4'd9)
                r_duty_cycle <= r_duty_cycle + 1;
            if (w_duty_dec && r_duty_cycle >= 4'd1)
                r_duty_cycle <= r_duty_cycle - 1;
        end
    end
```

```
    // 2. 10MHz PWM 신호 생성(0~9)
    always @(posedge clk, posedge reset) begin
        if (reset) begin
            r_pwm_counter <= 0;
        end else begin
        if (r_pwm_counter >= 4'd9)
            r_pwm_counter <= 0;
            else r_pwm_counter <= r_pwm_counter + 1;
        end
    end
    assign pwm_out = (r_pwm_counter < r_duty_cycle) ? 1'b1 : 1'b0;
    assign pwm_out_led = pwm_out;
    assign duty_cycle = r_duty_cycle;
endmodule
```

[fnd_controller.v]

```
`timescale 1ns / 1ps
module fnd_controller #(
    parameter BLINK_CYCLES = 100_000_000
)(input clk, reset, tick, [13:0]in_data, [1:0] motor_direction, output [3:0]an, [7:0]seg);

    localparam BLINK_WIDTH = (BLINK_CYCLES > 1) ? $clog2(BLINK_CYCLES) : 1;
    reg [BLINK_WIDTH-1:0] blink_count = 0;
    reg blink_on = 1'b1;
    // Toggle every second at 100 MHz: one second visible, one second blank.
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            blink_count <= 0;
            blink_on <= 1'b1;
        end else if (blink_count == BLINK_CYCLES - 1) begin
            blink_count <= 0;
            blink_on <= ~blink_on;
        end else begin
            blink_count <= blink_count + 1'b1;
        end
    end

    // Codes A/B are F/b; F is blank. SW[1:0] = {IN2, IN1}.
    wire [3:0] direction_digit = !blink_on ? 4'hf :
        (motor_direction == 2'b10) ? 4'ha :
        (motor_direction == 2'b01) ? 4'hb : 4'hf;

    wire [1:0] w_sel;
    fnd_digit_select u_fnd_digit_select(.clk(clk), .reset(reset), .tick(tick), .sel(w_sel));

    wire [3:0]w_d1, w_d10, w_d100, w_d1000;
    bin2bcd4_digit u_bin2bcd4_digit(.in_data(in_data), .d1(w_d1), .d10(w_d10),
.d100(w_d100), .d1000(w_d1000));

    fnd_digit_display u_fnd_digit_display(.sel(w_sel), .d1(w_d1), .d10(w_d10),
.d100(w_d100), .d1000(direction_digit), .an(an), .seg(seg));
endmodule

// logic select 1_digit displaying fnd every 1ms,
module fnd_digit_select(input clk, reset, tick, output reg[1:0] sel);
```

```
    always @(posedge clk or posedge reset) begin
        if(reset) begin
            sel <= 0;
        end else begin
            if(tick) begin
                sel <= sel + 1;
            end
        end
    end
endmodule

// in_data : bin size express to 9999 in 14bit fnd
// 0000 ~ 9999 /digit number 0 ~ 9 bcd 4bit express
module bin2bcd4_digit(input [13:0] in_data, output [3:0]d1, [3:0]d10, [3:0]d100, [3:0]d1000);

    assign d1 = in_data % 10;
    assign d10 = (in_data / 10) % 10;
    assign d100 = (in_data / 100) % 10;
    assign d1000 = (in_data / 1000) % 10;
endmodule

module fnd_digit_display(input [1:0]sel, [3:0]d1, [3:0]d10, [3:0]d100, [3:0]d1000, output reg [3:0]an, reg [7:0]seg);

    reg [3:0] bcd_data;

    always @(*) begin
        case(sel)
            2'b00: begin
                bcd_data = d1;
                an = 4'b1110;
            end
            2'b01: begin
                bcd_data = d10;
                an = 4'b1101;
            end
            2'b10: begin
                bcd_data = d100;
                an = 4'b1011;
            end
```

```
            2'b11: begin
                bcd_data = d1000;
                an = 4'b0111;
            end
            default: begin
                bcd_data = 4'b0000;
                an = 4'b1111;
            end
        endcase
    end

    always @(*) begin
        case(bcd_data)
            4'd0: seg = 8'b11000000;   // 0
            4'd1: seg = 8'b11111001;   // 1
            4'd2: seg = 8'b10100100;   // 2
            4'd3: seg = 8'b10110000;   // 3
            4'd4: seg = 8'b10011001;   // 4
            4'd5: seg = 8'b10010010;   // 5
            4'd6: seg = 8'b10000010;   // 6
            4'd7: seg = 8'b11111000;   // 7
            4'd8: seg = 8'b10000000;   // 8
            4'd9: seg = 8'b10010000;   // 9
            4'ha: seg = 8'b10001110;   // F
            4'hb: seg = 8'b10000011;   // b
            default: seg = 8'b11111111;  // all off
        endcase
    end
endmodule
```